

Submitted to

Kinetrexa Software Private Limited

Student Management System

Project Report

Prepared By

Bhargavi Kanojia

Application ID: KTS020260717887

Python Development Internship

Date of Submission: 30th July 2026

1. Introduction

Educational institutes and training centres routinely need a simple, reliable way to keep track of the students enrolled with them recording who they are, which course they belong to, and how to reach them without the overhead of a full database server. This project, submitted for the Python Development Internship at Kinetrexa Software Private Limited, is a console-based Student Management System that lets an administrator add, view, search, and delete student records through a menu-driven interface, with every record persisted to a local JSON file so data survives between runs.

The tool is built around Object-Oriented Programming principles: a Student class models a single record, a StudentManager class owns the in-memory list of students and all the CRUD (Create, Read, Update, Delete) logic, and a small storage module isolates the JSON read/write operations. `main.py` ties these pieces together behind a simple text menu, so each component can be tested, reused, or extended independently.

2. Objective

- Allow an administrator to add a new student record with ID, name, age, email, course name, and course ID.
- Prevent duplicate entries by rejecting a new student whose ID already exists.
- Display all currently stored student records in a readable, formatted list.
- Search for a specific student by their unique Student ID.
- Delete a student record by Student ID.
- Persist all student data to a JSON file so records are not lost when the program exits.
- Build the tool using Object-Oriented Programming with a modular file structure and basic exception handling for invalid or duplicate input.

3. Tools & Technologies Used

3.1 Core

- Python 3.x — core programming language
- `json` (built-in module) — reads and writes student records to `student.json`
- Object-Oriented Programming (OOP) — classes, constructors, encapsulation, static methods, and object serialization/deserialization

3.2 Development Tools

- Visual Studio Code — IDE
- Git & GitHub — version control and repository hosting

No third-party packages are required to run the application — the entire tool relies only on the Python standard library, which keeps setup effectively dependency-free.

4. Project Structure

The project follows a small, modular structure that separates the entry point, the data model, the business logic, and file persistence:

```
Student-Management-System/
├── main.py           # Entry point – menu-driven console interface
├── student.py        # Student class (data model)
├── manager.py        # StudentManager class – CRUD operations
├── storage.py        # JSON file read/write handling
├── student.json      # Persistent storage for student records
├── requirements.txt  # Project dependencies
└── README.md        # Project documentation
```

5. Features Implemented

5.1 Add Student

- Prompts the administrator for ID, name, age, email, course name, and course ID.
- Wraps the new record in a Student object before handing it to the manager.
- Rejects the addition and prints a clear message if a student with the same ID already exists (raised as a ValueError).
- Saves the updated list to student.json immediately after a successful add.

5.2 View Students

- Prints every stored record in a single formatted pass: ID, Name, Age, Email, Course Name, and Course ID.
- Prints a friendly "No students found" message when the list is empty instead of showing a blank screen.

5.3 Search Student

- Looks up a student by exact Student ID.
- Displays the full record if found, or a clear "not found" message otherwise.

5.4 Delete Student

- Searches for the student by ID, removes it from the in-memory list if found, and immediately re-saves student.json.
- Reports whether the deletion succeeded or the ID did not exist.

5.5 Update Student (manager-level capability)

- StudentManager exposes an update_student() method that can update any subset of a student's fields (name, age, email, course name, course ID) by ID and re-save the file — implemented in the manager module as a foundation for a future "Update" menu option.

5.6 Persistent JSON Storage

- All students are loaded from student.json into memory when the program starts.
- Every add or delete operation immediately re-writes the full list back to student.json, so the file always reflects the current state.

- A missing or corrupted JSON file is handled gracefully — the program starts with an empty student list instead of crashing.

6. Module-Wise Code Description

6.1 main.py — Entry Point

Creates a single StudentManager instance and runs a loop that prints a five-option menu (Add, View, Search, Delete, Exit) and dispatches to the matching manager method based on the administrator's choice:

```
manager = StudentManager()
while True:
    print("--- Student Management System ---")
    print("1. Add Student  2. View Students")
    print("3. Search Student  4. Delete Student  5. Exit")
    choice = input("Enter your choice: ")
    if choice == '1':
        # collect fields, build Student, manager.add_student(student)
    elif choice == '2':
        manager.view_students()
    elif choice == '3':
        manager.search_student(id)
    elif choice == '4':
        manager.delete_student(id)
    elif choice == '5':
        break
```

Invalid Student ID additions are caught with a try/except around add_student(), so a duplicate ID prints a clear message instead of crashing the program.

6.2 student.py — Student Data Model

Defines the Student class, which represents a single record and knows how to convert itself to and from a dictionary for JSON storage:

```
class Student:
    def __init__(self, id, name, age, email, course_name, course_id):
        self.id = id
        self.name = name
        self.age = age
        self.email = email
        self.course_name = course_name
        self.course_id = course_id

    def to_dict(self):
        return {'id': self.id, 'name': self.name, 'age': self.age,
                'email': self.email, 'course_name': self.course_name,
                'course_id': self.course_id}

    @staticmethod
    def from_dict(data):
        return Student(data['id'], data['name'], data['age'],
                       data['email'], data['course_name'], data['course_id'])
```

- `to_dict()` serializes a `Student` object into a plain dictionary so it can be written to JSON.
- `from_dict()` is a static factory method that rebuilds a `Student` object from a dictionary read out of `student.json`.

6.3 `manager.py` — `StudentManager` (Business Logic)

`StudentManager` owns the in-memory list of `Student` objects and every CRUD operation:

- `__init__()` calls `read_data()` from `storage.py` and rebuilds `Student` objects from the returned list using `Student.from_dict()`.
- `save()` converts every `Student` back to a dictionary with `to_dict()` and writes the whole list out via `write_data()`.
- `add_student(student)` loops through the existing list to check for a duplicate ID, raises `ValueError` if one is found, otherwise appends the record and calls `save()`.
- `view_students()` prints a formatted line per student, or a "No students found" message when the list is empty.
- `search_student(id)` returns the matching `Student` object or `None`.
- `delete_student(id)` reuses `search_student()` to locate the record, removes it from the list if found, and calls `save()`.
- `update_student(id, name=None, age=None, email=None, course_name=None, course_id=None)` updates only the fields that were actually passed in, then calls `save()`.

6.4 `storage.py` — JSON Persistence

A small, focused module that isolates all file I/O behind two functions:

```
FILE_NAME = 'student.json'

def read_data():
    try:
        with open(FILE_NAME, 'r') as file:
            return json.load(file)
    except (FileNotFoundError, json.JSONDecodeError):
        return []

def write_data(student):
    with open(FILE_NAME, 'w') as file:
        json.dump(student, file, indent=4)
```

- `read_data()` returns an empty list instead of crashing when `student.json` is missing or contains invalid JSON — this is the module's main piece of exception handling.
- `write_data()` always overwrites the file with the full, current list, pretty-printed with a 4-space indent for readability.

7. Student Information Stored

Each student record contains six fields:

- Student ID
- Student Name
- Student Age

- Student Email
- Course Name
- Course ID

Example JSON record as stored in student.json:

```
{
  "id": 101,
  "name": "Riya",
  "age": 23,
  "email": "riya841@gmail.com",
  "course_name": "B.Com.",
  "course_id": 13
}
```

8. Working / Execution Flow

1. Running python main.py starts the program and creates a StudentManager, which loads any existing records from student.json.
2. The console displays the main menu with five options: Add, View, Search, Delete, and Exit.
3. Choosing Add Student (1) prompts for ID, name, age, email, course name, and course ID, builds a Student object, and calls add_student() — which checks for a duplicate ID before saving.
4. Choosing View Students (2) calls view_students(), printing every currently stored record.
5. Choosing Search Student (3) prompts for an ID and calls search_student(), printing the matching record or a not-found message.
6. Choosing Delete Student (4) prompts for an ID and calls delete_student(), removing the record and re-saving the file if it exists.
7. Every Add or Delete operation immediately writes the updated list back to student.json via save(), so the file is always current.
8. Choosing Exit (5) prints a closing message and ends the loop; any other input is rejected with "Invalid choice. Please try again."

9. Sample Console Output

A typical run of the menu, followed by adding a student, looks like this:

```
--- Student Management System ---

1. Add Student
2. View Students
3. Search Student
4. Delete Student
5. Exit

Enter your choice: 1

Enter student ID: 101
Enter student name: Riya
Enter student age: 23
Enter student email: riya841@gmail.com
```

Enter course name: B.Com.
Enter course ID: 13

Student Riya added successfully.

10. Exception Handling

The application handles the failure points that are most likely to occur during everyday use, keeping the console output clear instead of showing a raw Python traceback:

- Duplicate Student IDs — `add_student()` raises a `ValueError` with a descriptive message, which `main.py` catches and prints instead of crashing.
- Missing `student.json` — `read_data()` catches `FileNotFoundError` and returns an empty list, so the program starts cleanly on first run.
- Empty or corrupted JSON file — `read_data()` also catches `json.JSONDecodeError` for the same reason, so a damaged file does not stop the program from launching.
- Invalid menu choices — any input outside 1–5 is caught by the final `else` branch and reported as "Invalid choice. Please try again."

Remaining hardening opportunity: numeric fields such as age and course ID are currently read as plain strings from `input()` without validation, so non-numeric entries are accepted rather than rejected up front.

11. Challenges Faced

- Designing a clean separation between the data model (`student.py`), the business logic (`manager.py`), and file persistence (`storage.py`) so each module has a single, well-defined responsibility.
- Preventing duplicate Student IDs from silently overwriting existing records, which required checking the full in-memory list before every add and raising a clear, catchable exception.
- Keeping `student.json` consistent with the in-memory list at all times by re-saving on every add and delete, without turning every read into a full file round-trip.
- Handling a first run where `student.json` does not yet exist, and a corrupted or empty file, without special-casing that logic outside of `storage.py`.
- Serializing and deserializing Student objects cleanly via `to_dict()` and `from_dict()` so the JSON file format stays simple and human-readable.

12. Future Enhancements

- Wire the existing `update_student()` method into the main menu as a fifth CRUD option.
- Add search by student name, not just by Student ID.
- Validate email format and numeric fields (age, course ID) at input time.
- Add course-wise filtering and listing of students.
- Add sorting of student records (by name, age, or ID).
- Migrate storage from a flat JSON file to a SQLite or MySQL database.

- Add a graphical user interface using Tkinter.
- Add a basic user login system to restrict access to the tool.

13. Conclusion

This project delivers a working, modular Student Management System that adds, views, searches, and deletes student records through a simple console menu, with every change persisted to a JSON file. Object-Oriented Programming principles a dedicated Student data model, object serialization via `to_dict()/from_dict()`, and a `StudentManager` class that encapsulates all CRUD logic — keep the codebase easy to read, test, and extend. Clear separation between `main.py`, `manager.py`, `student.py`, and `storage.py` means new features, such as wiring in the already-implemented `update_student()` method or adding input validation, can be layered on without disturbing the rest of the application.

14. GitHub Repository

Public GitHub Repository: [Code Link](#)